

pw-13th nov-3

codegen-

```
**npx playwright codegen --target=playwright-test --output=./tests/codegen1.spec.js  
--viewport-size=1920,1080 https://ineuron-courses.vercel.app/**
```

by extension pw will know if js or java etc.

Commands

Get Started

```
npx playwright codegen --help  
npx playwright codegen
```

Syntax

```
npx playwright codegen [options] [url]
```

Example1

```
npx playwright codegen --target test -o ./tests/file.spec.js urlToOpen
```

Example 2

```
npx playwright codegen --target test -o ./tests/file.spec.js --viewport-size=width,height url
```

Automate test.spec--

Example 1 where to save Example 2 set viewport

Test – means generate with test annotations.

—

configure in pw.

screenshot, video and tracer are switched on in chrome for every test run.

```

36
37  /* Configure projects for major browsers */
38  projects: [
39    {
40      name: 'chromium',
41      use: { ...devices['Desktop Chrome'],
42            screenshot: 'on',
43            video: 'on',
44            trace: "on",
45            headless: false
46          },
47    },
48  ],
49

```

```

//toContainText validation to check the text on webpage.
test("blank username and password", async function({page}){
  await page.goto("https://ineuron-courses.vercel.app/login")
  await page.locator("//button[normalize-space()='Sign in']").click()
  expect (await page.locator(".errorMessage")).toContainText("Email and Password is required")
})

```

codesnap.dev

file upload when there is input tag.

```
//when files have the input tag for them.
const {test, expect} = require("@playwright/test")

test("upload file which has input tag", async function({page}){

    //if we have multiple files, we can specify the path first.
    let file1 = "../tests/fixtures/file.js"
    let file2 = "../tests/fixtures/file.js"

    //go to the url where the file upload is present.
    await page.goto("C:/Users/karan/Desktop/playwrightbymukesh/fileupload.html")

    //to add one file use this.
    //setinputfiles has to be passed file.
    // await page.locator("//*[@id='myFile']").setInputFiles(file1)

    //send multiple files
    //send in array form
    await page.locator("//*[@id='myFile']").setInputFiles(file1, file2)

    //pause run for some time.
    await page.pause()

})
```

```
//when files do not have the input tag for them.
const {test,expect}=require("@playwright/test")

test("upload file which has no input tag", async function({page}){

  //specify path of the file.
  let file1="./tests/fixtures/file.js"

  //go to the url where the file upload is present.
  await page.goto("https://the-internet.herokuapp.com/upload")

  //use listeners for uploading files from computer.
  //on is the event we need to listen to.
  //filechooser is the event name, second param is callback function.
  //set files and pass one file or multiple files in form of array.
  page.on("filechooser", async (filechoose) =>{
    await filechoose.setFiles(file1)
    // await filechoose.setFiles([file1, file2])
  })

  //forcefully click on the link even if not clicked.
  await page.locator("#drag-drop-upload").click({force:true})

  //wait for time out requires ms.
  await page.waitForTimeout(10000)

  //pause does not require time.
  //pause opens inspector so we can run line by line or a complete run, for example like a debugger.
  await page.pause()

})
```

```

//Browser context- older way nowadays we have browser fixture and page fixture.
//we specify the browser on which we need to change specs.
//Customize specific spec file.

let {test, expect, chromium} = require("@playwright/test")

// const {test, expect, chromium} = require("@playwright/test")

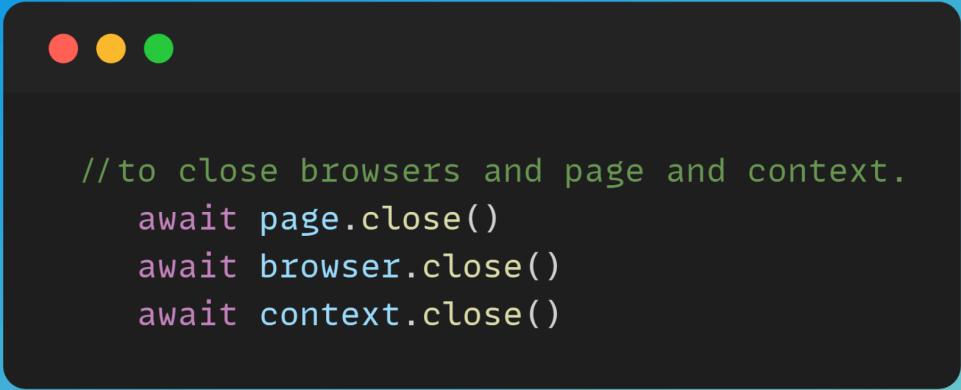
test("another way to launch browser with specific settings", async() => {

    //launch method.
    //pass in any property what you want to set in browser.
    //when we click launch we have all the options we can use,
    // similarly for newContext() and all others.
    //return type of launch is Browser.
    let browser = await chromium.launch({headless: false})
    //record videos and add them to the videos path in current project directory.
    let context = await browser.newContext({recordVideo: {dir: "./videos"}})
    //Creates a new page in the browser context.
    let page = await context.newPage()

    await page.goto("https://ineuron-courses.vercel.app/login")
    await page.getByPlaceholder("Email").click()
    await page.getByPlaceholder("Email").fill("m9@gmail.com")
    await page.getByPlaceholder("Password").fill("m9@gmail.com")
    await page.getByRole("button", {name: "Sign in"}).click();
    await expect(page).toHaveURL("https://ineuron-courses.vercel.app/")
    await page.getByRole("button", {name: "Sign out"}).click();
    await expect(page).toHaveURL("https://ineuron-courses.vercel.app/login")

})

```



```
//to close browsers and page and context.  
await page.close()  
await browser.close()  
await context.close()
```

codesnap.dev

—before all and after all function

```

//hooks for pre-running and post run.
let page
let browser
let context

//beforeall requires hook function
test.beforeAll(async()=>{
  browser =await chromium.launch({headless:false})
  context= await browser.newContext({recordVideo:{dir: "./videos"}})
  page=await context.newPage()
})

//afterAll requires hook function
test.afterAll(async()=>{
  await page.close()
  await browser.close()
  await context.close()
})

```

codesnap.dev

—before all, after all

```
let {test, expect, chromium} = require("@playwright/test")

//hooks for pre-running and post run.
let page
let browser
let context

//beforeall requires hook function
test.beforeAll(async() => {
  browser = await chromium.launch({headless: false})
  context = await browser.newContext({recordVideo: {dir: "./videos"}})
  page = await context.newPage()
})

//afterAll requires hook function
test.afterAll(async() => {
  await page.close()
  await browser.close()
  await context.close()
})
```

codesnap.dev

working with windows or tabs.


```

/**
 *
 */
const {test, expect} = require("@playwright/test")

//we directly pass browser fixture here.
test("handle window or tabs", async ({browser}) => {

    //creating browser context
    let browsercontext = await browser.newContext();

    //create new page
    let newpage = await browsercontext.newPage()

    await newpage.goto("https://ineuron-courses.vercel.app/login");

    //use promise to wait for the page to load and then perform the click operation.
    let [newpage1] = await Promise.all(
        [
            browsercontext.waitForEvent("page"),
            //click on the twitter link on the login page of ineuron.
            newpage.locator("//a[@href='https://twitter.com/iNeuronAi']//img").click()
        ]
    )

    //wait for twitter url to open
    const url = await newpage1.url()
    console.log("url is " + url)

    //lets use some words from ui in ineuron login page
    //we want to capture ineuronAI
    const newwordfromurl = url.split("com")[1].substring(1)

    //we dont have to go back to older page like selenium
    //we only have to use the name which we used to refer to original page
    await newpage.locator("//input[@id='email1']").click()
    await newpage.locator("//input[@id='email1']").type(newwordfromurl)

})

```

codesnap.dev

—All js alerts auto handled in pw and we can also do it manually.